

1. First-Come, First-Served (FCFS)

Pseudo Code

```
# FCFS
def fcfs_scheduler(processes):
    #
    now = 0
    for p in processes:
        if now < p.arrival_time:
            now = p.arrival_time # CPU
        run_process(p) #
        now += p.burst_time #
```

First-Come, First-Served (FCFS) is a scheduling algorithm where processes are executed in the order they arrive at the CPU. The process with the earliest arrival time is served first, and the process with the longest burst time is served last.

Case Study

Process	Arrival Time	Burst Time
P1	0	15
P2	0	7
P3	0	11
P4	0	20

At t=0, all processes are ready. Tie-break is based on arrival time. P3 → P4 → P1 → P2.

Gantt Chart for the Schedule

P3		P4		P1		P2
0	11	31		46		53

FCFS is a non-preemptive scheduling algorithm. The process with the earliest arrival time is served first, and the process with the longest burst time is served last.

11/11/2019

0~11	P3	1 (burst=11)
11~31	P4	2 (burst=20)
31~46	P1	3 (burst=15)
46~53	P2	4 (burst=7)

11/11/11

Process	Burst		 (TAT)	
P3	11	11	11	0
P4	20	31	31	11
P1	15	46	46	31
P2	7	53	53	46

■■■■ ■■■■ ■■ ■■ P2(burst=7)■ ■■ ■■■■ ■■■■ ■■ ■■■■ 46ms■ ■■■■. ■■ FCFS■
 ■■■■ ■■■■ **Convoy Effect**■, ■ ■■■■ ■■ ■■■■ ■■ ■■■■ ■■ ■■■■ ■■■■. ■■
 ■■■■ ■■ ■■ ■■ ■■ ■■ ■■ ■■ ■■ ■■ ■■, ■■ ■■ ■■ ■■■■
 ■■■■(P2→P3→P1→P4)■ ■■■■, ■ ■■ ■■ ■■■■ ■■■■(P4→P1→P3→P2)■ ■■■■.

FCFS 是 最 先 入 最 先 出 的 排 队 方 式 。 在 这 种 方 式 下 ， 每 个 任 务 都 按 照 到 达 系 统 的 顺 序 来 执 行 ， 先 到 先 服 务 。 这 种 方 式 的 优 点 是 简 单 易 实 施 ， 但 是 它 的 缺 点 是 不 能 保 证 任 务 的 执 行 时 间 最 短 ， 因 此 它 的 效 率 相 对 较 低 。

11

- $\{a_i, b_i, c_i\}$: arrival time, CPU burst time, c_i tie-break ($c_i = 0$ if $a_i = b_i$) for $i = 1, \dots, n$.
- $\{a_i, b_i, c_i\}$: a_i arrival time, b_i CPU burst time, c_i tie-break ($c_i = 0$ if $a_i = b_i$) for $i = 1, \dots, n$.
Convoy Effect $\{a_i, b_i, c_i\}$ for $i = 1, \dots, n$.
- $\{a_i, b_i, c_i\}$: a_i arrival time, b_i CPU burst time, c_i tie-break ($c_i = 0$ if $a_i = b_i$) for $i = 1, \dots, n$.
CPU $\{a_i, b_i, c_i\}$ for $i = 1, \dots, n$.

$\{a_i, b_i, c_i\}$

- $\{a_i, b_i, c_i\}$, $\{a_i, b_i, c_i\}$.
- $\{a_i, b_i, c_i\}$ for $i = 1, \dots, n$.

$\{a_i, b_i, c_i\}$

- CPU $\{a_i, b_i, c_i\}$ for $i = 1, \dots, n$, I/O $\{a_i, b_i, c_i\}$ for $i = 1, \dots, n$ (Convoy Effect) $\{a_i, b_i, c_i\}$ for $i = 1, \dots, n$.

- **Shortest Job First (SJF) Scheduling:** Selects the process with the shortest burst time for execution.

2. Shortest Job First (SJF)

Pseudo Code

```
# SJF Scheduling Algorithm
def sjf_scheduler(processes):
    # Initialize variables
    now = 0
    while processes:
        # Find processes available at current time
        available_processes = [p for p in processes if p.arrival_time <= now]
        if not available_processes:
            now += 1 # CPU idle, move to next time unit
            continue
        # Select process with shortest burst time
        next_process = min(available_processes, key=lambda p: p.burst_time)
        run_process(next_process) # Execute process
        now += next_process.burst_time # Move to next time unit
        processes.remove(next_process) # Remove process from queue
```

■ ■

Shortest Job First (SJF) scheduling algorithm selects the process with the shortest burst time for execution. In this example, the CPU will execute processes in the order: P3, P4, P2, P1.

Case Study

Process	Arrival Time	Burst Time
P1	0	10
P2	4	13
P3	8	7
P4	10	5

Gantt Chart for the Schedule

P1			P4	P3		P2		
0	10		15		22		35	

■■■ ■■■ ■■■ ■■■

■ SJF ■■■■ ■■■■■■ ■■■ ■■■■■■ ■■ ■■■■ ■■■ ■■■■ ■■ CPU■ ■■■■■■
 ■■■■■ ■■■■ ■■■■■■.

■■ ■■

- **t=0:** P1■ ■■■■ ■■■■■ P1(burst=10)■ ■■■■■■.
- **t=10:** P1 ■■■. ■■ ■■■■ ■■■■■: P2(arr=4, burst=13), P3(arr=8, burst=7), P4(arr=10, burst=5). SJF■ ■■■ ■■ P4(burst=5)■ ■■■■■■.
- **t=15:** P4 ■■■. ■■■ P2(burst=13), P3(burst=7). P3(burst=7)■ ■■■■■■ P3■ ■■■■■■.
- **t=22:** P3 ■■■. P2(burst=13)■ ■■■ ■■■■■■.

■■ ■■

Process	Arrival	Burst	■■ ■■■	■■ ■■■ (TAT)	■■ ■■■
P1	0	10	10	10	0
P4	10	5	15	5	0

P3	8	7	22	14	7
P2	4	13	35	31	18

P4 进程在 CPU 上等待时间 0ms。进程 P2(burst=13) 在 SJF 调度算法下，其等待时间 (18ms) 是多少？SJF 调度算法是否存在 Starvation 问题？

进程 P3 在 SJF 调度算法下，其等待时间是多少？进程 P4 在 SJF 调度算法下，其等待时间是多少？

进程 P2

- 进程 P2 的等待时间 (arrival time, CPU burst time) 是多少？SJF 调度算法是否存在 Starvation 问题？

- 进程 P3 的等待时间 (arrival time, CPU burst time) 是多少？SJF 调度算法是否存在 Starvation 问题？

- 进程 P4 的等待时间 (arrival time, CPU burst time) 是多少？SJF 调度算法是否存在 Starvation 问题？

进程 P2

- 进程 P3 的等待时间 (arrival time, CPU burst time) 是多少？SJF 调度算法是否存在 Starvation 问题？

- CPU 利用率 (arrival time, CPU burst time) 是多少？SJF 调度算法是否存在 Starvation 问题？

■ ■

- ■■■■■■ ■■■■ ■■■■ ■■■■ ■■■■ ■■■■■■.
- ■ ■■■■ ■■■■ ■■ ■■■■■■ ■■■■■■ ■■■■■■ ■■(Starvation)■ ■■■■ ■ ■■■■■■.

3. Shortest Remaining Time First (SRTF)

Pseudo Code

```
# SRTF ■■■■■■ ■■■■■■ ■■
def srtf_scheduler(processes):
    # ■■■■■■ ■■ ■■■■ ■■■■ ■■■■ ■■■■■■ ■■
    now = 0
    while processes:
        # ■■ ■■■■■■ ■■■■ ■■■■■■ ■■■■ ■■ ■■■■ ■■■■ ■■ ■■ ■■■■■■ ■■
        available_processes = [p for p in processes if p.arrival_time <= now]
        if not available_processes:
            now += 1 # CPU■ ■■ ■■■■ ■■, ■■ ■■■■■■ ■■
            continue
        next_process = min(available_processes, key=lambda p: p.remaining_time)
        run_process(next_process) # ■■■■■■ ■■
        now += next_process.remaining_time # ■■ ■■ ■■■■■■
        processes.remove(next_process) # ■■■■ ■■■■■■ ■■
```

■ ■

Shortest Remaining Time First (SRTF) ■■■■■■ ■■■■■■ ■■ ■■■■■■ ■■■■ ■■■■■■ ■■■■ ■■
■■■■ ■■■■ ■■ ■■ ■■■■■■ ■■■■■■ CPU■ ■■■■■■ ■■■■■■. ■ ■■■■■■ ■■■■ ■■■■ ■■■■■■.

Case Study

- 在系统资源分配中，饥饿（Starvation）是指长期无法获得所需资源的状态。

- 在数据库事务调度中，饥饿（Starvation）是指事务长时间无法获得锁资源，导致无法完成。

二、

- 在操作系统中，饥饿（Starvation）是指进程长时间无法获得CPU资源。

- 在数据库事务调度中，饥饿（Starvation）是指事务长时间无法获得锁资源，导致无法完成。

三、

- 在操作系统中，饥饿（Starvation）是指进程长时间无法获得CPU资源。

- 在数据库事务调度中，饥饿（Starvation）是指事务长时间无法获得锁资源，导致无法完成。

4. Priority Scheduling

Pseudo Code

```
# Priority Scheduling
def priority_scheduler(processes):
    #
    now = 0
    while processes:
        #
        available_processes = [p for p in processes if p.arrival_time <= now]
        if not available_processes:
            now += 1 # CPU
            continue
        next_process = max(available_processes, key=lambda p: p.priority)
        run_process(next_process) #
        now += next_process.burst_time #
        processes.remove(next_process) #
```

Priority Scheduling
CPU

Case Study

Process	Arrival Time	Burst Time	Priority
P1	0	5	4
P2	2	12	1
P3	3	18	3
P4	5	9	2

Priority 1 =

Gantt Chart for the Schedule

P1	P2	P4	P3	P1
2 0	14	23	41 44	

Process	Priority	Burst	Wait Time	Turnaround Time (TAT)	Response Time
P2	1 (High)	12	14	12	0
P4	2	9	23	18	9
P3	3	18	41	38	20
P1	4 (Low)	5	44	44	39

P2's $t=2$ ms burst is executed first (prio=1) as P1's 44ms burst (prio=4) is not yet started. After P2's burst, P4's 9ms burst is executed (prio=2). Then P3's 18ms burst is executed (prio=3). Finally, P1's 5ms burst is executed (prio=4). This illustrates Priority Scheduling where processes are executed based on their priority. Starvation can occur if a process with a low priority never gets a chance to execute.

Priority Scheduling is a scheduling algorithm that executes processes based on their priority. It is a non-preemptive algorithm, meaning that once a process starts executing, it runs to completion. The process with the highest priority is executed first. If two processes have the same priority, they are executed in the order they arrived. This algorithm is simple to implement but can suffer from starvation if a process with a low priority never gets a chance to execute.

•

- **Arrival Time, CPU Burst Time, Priority:** arrival time, CPU burst time, priority. These are the attributes used to schedule processes.

- **Starvation:** occurs when a process with a low priority never gets a chance to execute. This can happen in priority scheduling if the system is always executing high-priority processes.

- **Starvation:** occurs when a process with a low priority never gets a chance to execute. This can happen in priority scheduling if the system is always executing high-priority processes.



- **Process with highest response ratio is selected.**
- **When a process enters the ready queue, its response ratio is calculated as the ratio of its waiting time plus its execution time to its execution time. When the process is selected, its response ratio is reset to 1.**



- **Starvation (Starvation) can occur if a process is never selected.**
- **Starvation can be avoided by using a time limit for each process.**

5. HRRN (Highest Response Ratio Next)

Pseudo Code

```
# HRRN Scheduler Pseudo Code
def hrrn_scheduler(processes):
    # Initialize a queue with all processes
    now = 0
```

```

while processes:
    # ■■■ ■■■■■ ■■■ ■■■■■ ■■■ ■■ ■■■ ■■ ■■ ■■■■■ ■■
    available_processes = [p for p in processes if p.arrival_time <= now]
    if not available_processes:
        now += 1 # CPU■■ ■■ ■■■ ■■, ■■ ■■■■■ ■■
        continue
    next_process = max(available_processes, key=lambda p: (now - p.arrival_time + p.burst_time) / p.burst_time)
    run_process(next_process) # ■■■■■ ■■
    now += next_process.burst_time # ■■ ■■ ■■■■■
    processes.remove(next_process) # ■■■ ■■■■■ ■■

```

■■■

HRRN (Highest Response Ratio Next) ■■■■■ ■■■■■■■ ■■ ■■■■■ ■■■ ■■■■■ ■■■ ■■ ■■■■
 ■■ ■■ ■■■■■■■ ■■■■■ CPU■■ ■■■■■ ■■■■■■■. ■■ ■■■■ (■■■ ■■ + ■■■■ ■■) / ■■■■ ■■■■■
 ■■■■■■■. ■ ■■■■■■■ ■■■■■ ■■■ ■■■■■■■.

Case Study

Process	Arrival Time	Burst Time
P1	0	10
P2	1	28
P3	2	6
P4	3	4

Gantt Chart for the Schedule

P1		P4	P3	P2			
0	10	14	20				48

■■■ ■■■ ■■ ■■■

■ ■■■■■ HRRN ■■■■■ ■■■■■■■ ■ ■■ ■■■■■■■ ■■ ■■(HRR)■ ■■■■■ CPU■■ ■■■■■
 ■■■■■ ■■■ ■■■■■■■. ■■ ■■ = (■■■ ■■ + ■■■■ ■■) / ■■■■ ■■.

t=0: P1 ■■ ■■ → P1(burst=10) ■■, t=10 ■■.

t=10 — HRRN ■■ (P2, P3, P4 ■■ ■■)

Process	■ ■	Burst	■ ■ = (■ ■) + Burst) / Burst
P2	10 − 1 = 9	28	(9 + 28) / 28 = 37/28 ≈ 1.32
P3	10 − 2 = 8	6	(8 + 6) / 6 = 14/6 ≈ 2.33
P4	10 − 3 = 7	4	(7 + 4) / 4 = 11/4 = 2.75 ← ■ ■

→ P4 ■■■, t=10~14 ■■■. P4 ■■■.

t=14 — HRRN ■■■■ (P2, P3 ■■■ ■)

Process	■ ■	Burst	■ ■
---------	--	-------	--

P2	$\frac{14 - 1}{13} = 13$	28	$\frac{(13 + 28)}{28} = \frac{41}{28} \approx 1.46$
P3	$\frac{14 - 2}{12} = 12$	6	$\frac{(12 + 6)}{6} = \frac{18}{6} = 3.0$ ← ■ ■

→ P3 ■■■, t=14~20 ■■■. P3 ■■■. ■■■ P2■ ■■■ t=20~48 ■■■.

■■■ ■■■

Process	Burst	■■■ ■■■	■■■ ■■■ (TAT)	■■■ ■■■
P1	10	10	10	0
P4	4	14	11	7
P3	6	20	18	12
P2	28	48	47	19

HRRN■■ ■■■ ■■■ ■■■ ■■■ ■■■ ■■■ ■■■ ■■■ Starvation■ ■■■■■■. P2■ burst=28■■■ ■■■ ■■■■, ■■■■■ ■■■ ■■■ ■■■ ■■■ ■■■■ ■■■ t=20■ CPU■ ■■■■■■■■. ■■■ SJF■■■■ ■■■ ■■■■ ■■■ ■■■■ P2■ ■■■■ ■■■■■ ■■■■■, HRRN■■■■ ■■■ ■■■ ■■■■ ■■■■ ■■■■.

■■■■ HRRN■ ■■■ ■■■ ■■■ ■■■ ■■■ ■■■ ■■■ ■■■ ■■■ ■■■■ ■■■■ ■■■ ■■■■■. ■■■■■■ SJF■■■ ■■■■■ ■■■■■■■■ ■■■ ■■■■■ ■■■■ ■■■■ ■■■■■■.

■■■

- t_{arr} arrival time, t_{burst} CPU burst time. t_{wait} waiting time.
- t_{wait} waiting time: time from arrival to start of execution. $t_{starvation}$ Starvation time: time from arrival to start of execution.
- t_{wait} , $t_{starvation}$: time from arrival to start of execution, t_{wait} waiting time, $t_{starvation}$ Starvation time.

•

- t_{wait} waiting time, $t_{starvation}$ Starvation time.

- t_{wait} waiting time, $t_{starvation}$ Starvation time.

•

- t_{wait} waiting time, $t_{starvation}$ Starvation time.

- [REDACTED]

6. Round Robin (RR)

Pseudo Code

```
# RR [REDACTED]
def rr_scheduler(processes, time_quantum):
    # [REDACTED]
    now = 0
    queue = []
    while processes or queue:
        # [REDACTED]
        for p in processes:
            if p.arrival_time <= now:
                queue.append(p)
        processes = [p for p in processes if p.arrival_time > now]

        if not queue:
            now += 1 # CPU [REDACTED]
            continue

        current_process = queue.pop(0) # [REDACTED]
        run_time = min(time_quantum, current_process.remaining_time)
        run_process(current_process, run_time) # [REDACTED]
        now += run_time # [REDACTED]
        current_process.remaining_time -= run_time # [REDACTED]

        if current_process.remaining_time > 0:
            queue.append(current_process) # [REDACTED]
```

[REDACTED]

Round Robin (RR) [REDACTED] (time quantum) [REDACTED] CPU [REDACTED] [REDACTED]. [REDACTED]

Case Study

Process	Arrival Time	Burst Time
P1	0	10

Case Study

P1	0	8	Q0 (SRTF, ■■■■■)
P2	1	4	Q0 (SRTF, ■■■■■)
P3	0	6	Q1 (HRRN, ■■■■■)
P4	2	4	Q1 (HRRN, ■■■■■)

| Q0■ ■■ ■■ ■■ Q1■ CPU■ ■■■■■. Q0■ ■■■■ SRTF, Q1■ ■■■■ HRRN■ ■■■■■.

Gantt Chart for the Schedule

Q0:P1		Q0:P2		Q0:P1		Q1:P4		Q1:P3	
0	1			5	12	16		22	

Queue	Execution Order	Time Marks
Q0	P1 → P2 → P1	0, 1, 5, 12
Q1	P4 → P3	12, 16, 22

■ ■ ■ ■ ■ ■ ■ ■ ■ ■

■ ■■■■■ Multilevel Queue ■■■■ ■■■■■ ■ ■■ ■(Q0: SRTF, Q1: HRRN)■ ■■■■ ■ ■■
 ■■■■■ CPU■ ■■■■ ■■■■ ■■■ ■■■■■.

Q0 (SRTF) ■ ■ ■ ■

- t=0: Q0■ P1(rem=8)■ ■■■■■ P1■ ■■■■■.
- t=1: P2(rem=4)■ Q0■ ■■■■■. SRTF ■■■ ■■ ■■ ■■■ ■■■ ■ ■■ P2(rem=4 < P1 rem=7)■ P1■ ■■■■■.

- t=5: P2 (burst=4). Q0 P1(rem=7) P1.
- t=12: P1. Q0 Q1 CPU.

Q1 (HRRN)

Q0 t=12 HRRN.

Process	(t=12)	Burst Time	= (+ Burst) / Burst
P3	12 – 0 = 12	6	(12 + 6) / 6 = 3.0
P4	12 – 2 = 10	4	(10 + 4) / 4 = 3.5

P4(3.5).

- t=12 ~ t=16: P4 (burst=4),.

- t=16 ~ t=22: P3 (burst=6),

Process		(TAT)	
P1	12	12 - 0 = 12	12 - 8 = 4
P2	5	5 - 1 = 4	4 - 4 = 0
P3	22	22 - 0 = 22	22 - 6 = 16
P4	16	16 - 2 = 14	14 - 4 = 10

Q0 (P1, P2) SRTF, Q1 (P3, P4) Q0, HRRN, FCFS P4 P3 Starvation.

, , , .

- `struct task { int arrival_time, CPU_burst_time, priority; }` arrival time, CPU burst time, priority. `task_t` is a pointer to a `task` structure.

- `task_t *tasks;` array of pointers to task structures. `tasks[i]` is a pointer to the `i`-th task structure.

- `int n;` number of tasks. `tasks[n]` is a pointer to the `n`-th task structure.

`main`

- `task_t *tasks;` array of pointers to task structures.

- `int n;` number of tasks.

`main`

- `task_t *tasks;` array of pointers to task structures.

- Starvation ☐ ☐ ☐ ☐.

8. Multi-Level Feedback Queue

```
# MLFQ ■■■■ ■■■■ ■■
def mlfq_scheduler(processes):
    # Q0, Q1, Q2■ RR / Q3■ FCFS
    time_quantum = [2, 4, 8, None]
    boost_interval = 100 # ■■■■ ■■ ■■■■ ■■■
    queues = [Queue() for _ in range(4)]
    now = 0

    # ■■ ■■■■ ■■■ ■■■ ■■
    for p in processes:
        p.level = 0
        p.used_in_level = 0 # Time Accounting ■■ ■■
        queues[0].enqueue(p)

    while has_runnable_or_waiting_process(processes):
        # Priority Boosting: ■■ ■■ ■■
        if now > 0 and now % boost_interval == 0:
            for p in all_ready_processes_in_any_queue(queues):
                p.level = 0
                p.used_in_level = 0
                queues[0].enqueue(p)

        # ■■ ■■ ■■■■ ■■■■ ■■ ■ ■■
        level = highest_non_empty_queue_index(queues)
        if level is None:
            now += 1
            continue

        p = queues[level].dequeue()

        if level < 3:
            # RR ■■: ■■ ■ ■■■■ ■■
            ran, event = run_process_for_quantum(p, time_quantum[level])
        else:
            # ■■■ ■: FCFS (■■ ■■ I/O■■ ■■ ■■)
            ran, event = run_process_fcfs(p)

        now += ran

    if event == "FINISHED":
```

```
continue

if event == "IO_BLOCK":
    # I/O
    p.level = level
    p.used_in_level += ran # Time Accounting:
    enqueue_after_io_completion(p, queues[p.level])
else: # event == "QUANTUM_EXPIRED"
    p.used_in_level += ran
    #
    if level < 3 and p.used_in_level >= time_quantum[level]:
        p.level = level + 1
        p.used_in_level = 0
    else:
        p.level = level
    queues[p.level].enqueue(p)
```

(Multi-Level Feedback Queue, MLFQ) is a scheduling algorithm that classifies processes into different levels based on their behavior. Processes that are CPU-bound (high CPU usage) are moved to higher levels (lower priority), while processes that are I/O-bound (low CPU usage, high I/O) are moved to lower levels (higher priority). This helps in providing better response time for interactive processes. - MLFQ is a non-preemptive scheduling algorithm. - MLFQ uses a quantum of time to move processes between levels. - MLFQ can be implemented with or without priority boosting. CPU-bound processes are moved to higher levels (lower priority) and I/O-bound processes are moved to lower levels (higher priority). CPU-bound processes are moved to higher levels (lower priority) and I/O-bound processes are moved to lower levels (higher priority).

Case Study

Process	Arrival Time	Total CPU Burst	
P1	0	12	CPU-bound,
P2	1	1	interactive
P3	2	6	

Q0: RR(2ms), Q1: RR(4ms), Q2: RR(8ms), Q3: FCFS. Q0: CPU burst, I/O Priority Boosting.

Gantt Chart for the Schedule

5~9	P1	Q1	4ms ■■ ■■, rem=6 → Q2■ ■■
9~13	P3	Q1	4ms ■■ ■ ■■
13~19	P1	Q2	6ms ■■ ■■ ■ ■■

■■ ■■

Process	■■ ■■	■■ ■■ (TAT)	■■ ■■
P2	3	3 - 1 = 2	2 - 1 = 1
P3	13	13 - 2 = 11	11 - 6 = 5
P1	19	19 - 0 = 19	19 - 12 = 7

■■ ■■■ P2■ ■■■ ■ Q0■■ ■■ ■■■■ ■■ ■■ ■■■■ ■■■■. ■■ CPU■ ■■ ■■■■ P1■
Q0, Q1, Q2■ ■■ ■■■■ ■■■■. ■■■ MLFQ■ ■■■■, interactive ■■■■ ■■ ■■■■ ■■
CPU-bound ■■■■ ■■ ■ ■■ ■■■■ ■■■■.

■ ■■■■ ■■■■ ■■■■, ■■ MLFQ■■■ ■■ ■■■ ■ ■■ ■■ I/O■ ■■■■ ■■■ ■■ ■■■■
■■■■■ ■■■ ■ ■■, ■■ ■■■ ■■■■ ■■ ■■■ Priority Boosting■ ■■■■. ■■ Time
Accounting■ ■■■■ ■■ ■■■■ ■■■■■ I/O■ ■■■ ■■ ■■ ■■ ■■■■ Gaming the
Scheduler ■■■■ ■■■ ■ ■■■■.

MLFQ는 우선 순위가 낮은 프로세스를 먼저 실행하고, 우선 순위가 높은 프로세스를 나중에 실행하는 방식이다. 이는 프로세스의 우선 순위를 낮추고, 프로세스의 실행 시간을 늘리는 방식이다. 이는 프로세스의 실행 시간을 늘리는 방식이다.

이러한 방식은

- Gaming the Scheduler 문제 : 프로세스가 자신의 우선 순위를 낮추고, I/O를 요청하여 자신의 실행 시간을 늘리는 것을 방지하는 방법을 찾아야 한다.

- 시간 회계(Time Accounting) : 프로세스가 자신의 실행 시간을 기록하고, I/O를 요청할 때 자신의 실행 시간을 기록하고, I/O를 요청할 때 자신의 실행 시간을 기록하고, I/O를 요청할 때 자신의 실행 시간을 기록한다.

이러한 방식은

- 프로세스의 실행 시간을 기록하는 방법 : 프로세스의 실행 시간을 기록하고, I/O를 요청할 때 자신의 실행 시간을 기록하고, I/O를 요청할 때 자신의 실행 시간을 기록한다.

- 프로세스의 실행 시간을 기록하는 방법 : 프로세스의 실행 시간을 기록하고, CPU를 사용하여 프로세스의 실행 시간을 기록하고, CPU를 사용하여 프로세스의 실행 시간을 기록한다.

- 프로세스의 실행 시간을 기록하는 방법 : Time Accounting, Gaming, Starvation, Priority Boosting 등 다양한 방법을 사용하여 프로세스의 실행 시간을 기록한다.

■ ■

- CPU ■■■■ I/O ■■■■ ■■■■ ■■■■ ■■■■ ■■■■ ■■■■.
- ■■■■ ■■ ■■ ■■ ■■■■ ■■■■ ■■■■ ■■■■ CPU ■■■■ ■■■■.
- ■■■■ ■■■■ ■■ ■■ ■■ ■■■■ ■■ ■■ ■■■■ ■■ ■■ ■■ ■■ ■■■■.
- ■■■■ ■■■■ ■■■■ ■■■■ ■■■■ ■■■■ ■■■■.

■ ■

- ■■■■ ■■■■, ■■ ■■ ■■ ■■■■ ■■■■.
- ■■■■ ■■ ■■ ■■■■ ■■■■ ■■■■, ■■■■ ■■ ■■ ■■■■ ■■ ■■■■ ■■■■ ■■■■.
- ■■■■ ■■■■ ■■ ■■ ■■■■ ■■■■ ■■■■ ■■■■ ■■■■ ■■■■.

9. Fair Share Scheduling

Pseudo Code

```
# Fair Share Scheduling
def fair_share_scheduler(processes, user_limits):
    # user_limits: {user_id: max_cpu_share}
    now = 0
    while has_runnable_or_waiting_process(processes):
        # Calculate CPU usage for each user
        user_usage = calculate_user_cpu_usage(processes)

        # Find available processes based on user limits
        available_processes = [p for p in processes if p.arrival_time <= now and user_usage[p.user_id] < user_limits[p.user_id]]

        if not available_processes:
            now += 1 # CPU is busy, skip to next time unit
            continue

        next_process = select_fair_process(available_processes, user_usage, user_limits)
        run_process(next_process) # Run the selected process
        now += next_process.burst_time # Move to the next time unit
```

...

Fair Share Scheduling ensures that each user's processes receive a fair share of the CPU based on their limits. In this example, UserA's processes (P1, P2) and UserB's processes (P3, P4) are scheduled in a way that respects their respective CPU share limits (60% for UserA, 40% for UserB) and their burst times.

Case Study

Process	User	Arrival Time	Burst Time
P1	UserA	0	6
P2	UserA	0	4
P3	UserB	0	3
P4	UserB	0	5

CPU usage: UserA 60%, UserB 40% (3:2) / Time Quantum = 2ms
Scheduling: UserA gets 3 units (6ms), UserB gets 2 units (4ms) using Round Robin (RR) scheduling.
Each process runs for its burst time divided by the time quantum.

Gantt Chart for the Schedule

UA:P1	UA:P2	UB:P3	UA:P1	UB:P4	UA:P2	UA:P1	UB:P3	UB:P4	
2	0 4	6	8	10	12	14	15	18	

User	Execution Order	Time Marks
UserA	P1 → P2 → P1 → P2 → P1	0, 2, 6, 10, 12, 14
UserB	P3 → P4 → P3 → P4	4, 8, 14, 15, 18

■■■ ■■■ ■■■ ■■■ ■■■

[illegible]

11111111 11111111

■■■■ ■■ 3:2 ■■■■■ ■ ■■■■■■ UserA ■ 3■■■(■ 6ms), UserB ■ 2■■■(■ 4ms) ■ ■■■■■■■. ■■■■ ■■■■■ ■■ ■■■■ ■■ ■ RR ■■ ■■■■■ ■■■■■.

■■■ 1 (t=0 ~ t=10)

■ ■	■ ■ ■ ■ ■ ■	■ ■ ■	■ ■
0~2	P1	UserA	UserA ■ ■ ■ ■ ■
2~4	P2	UserA	UserA ■ ■ ■ ■ ■ (UserA ■ RR)
4~6	P3	UserB	UserB ■ ■ ■ ■ ■

6~8	P1	UserA	UserA ■ ■■ ■■ (UserA ■ RR)
8~10	P4	UserB	UserB ■ ■■ ■■ (UserB ■ RR)

→ t=10 ■■ ■■ ■■■■ UserA 6ms, UserB 4ms■■ ■■■■ 60% : 40%■■ ■■■■■■.

■■■■ 2 (t=10 ~ t=14)

■■	■■ ■■■■	■■■	■■
10~12	P2	rem: 2→0	P2 ■■
12~14	P1	rem: 2→0	P1 ■■

→ t=14 ■■■■ UserA■■ ■■ ■■■■■■ ■■■■■■, ■■ ■■ CPU■■ UserB■■ ■■■■■■.

UserB ■■ ■■ (t=14 ~ t=18)

■■	■■ ■■■■	■■	■■
14~15	P3	rem: 1→0	P3 ■■
15~18	P4	rem: 3→0	P4 ■■

■■ ■■

Process	User	Burst	■■ ■■	■■ ■■ (TAT)	■■ ■■
P1	UserA	6	14	14	8

P2	UserA	4	12	12	8
P3	UserB	3	15	15	12
P4	UserB	5	18	18	13

UserA arrived at t=14ms, CPU burst time is 4ms. UserB arrived at t=10.25ms, CPU burst time is 3ms. UserA arrived at t=14ms, CPU burst time is 4ms. UserB arrived at t=10.25ms, CPU burst time is 3ms. Fair Share Scheduling algorithm is used to schedule the processes. The CPU utilization is 100% and the system is in a steady state.

Fair Share Scheduling algorithm is used to schedule the processes. The CPU utilization is 100% and the system is in a steady state.

The

- The arrival time, CPU burst time, user_id, and quota of the processes are given in the table below.

- The arrival time, CPU burst time, user_id, and quota of the processes are given in the table below.

- The arrival time, CPU burst time, user_id, and quota of the processes are given in the table below.

The

- The arrival time, CPU burst time, user_id, and quota of the processes are given in the table below.

- **Each process has a fixed priority.**

- **Each process has a fixed priority.**

•

- **Each process has a fixed priority, and CPU quota is used to limit the amount of time a process can execute.**

- **Each process has a fixed priority, and CPU quota is used to limit the amount of time a process can execute. trade-off is made between priority and quota.**

- **Each process has a fixed priority, and CPU quota is used to limit the amount of time a process can execute. trade-off is made between priority and quota.**

10. Multiple-Processor Scheduling

Pseudo Code

```
// Multiple-Processor(Scheduler)
// - Scheduler: Scheduler + Scheduler/Processor
while (System_Is_Running):
    for each Processor in Processors:
        if (Processor.is_idle()):
            // 1) Scheduler Scheduler (Affinity Scheduler)
```

11

- **11/12 vs 11/11 11/10 11/9 11/8 11/7 11/6**

- ■■■■ NUMA ■■ ■■■ ■■ ■■■ ■■■■■ ■■

Task	Arrival	CPU Burst	Queue Size [][]([][])	Schedule Order [][]
T1	0	18	Socket 0	[][][[][]

- `task_struct(pte)` can be used to find the task's pte.
- `task_struct` can be used to find the task's tail latency.
- NUMA-aware `task_struct` can be used to find the task's pte.

■

- `task_struct`, `task_struct(pte/NUMA)` can be used to find the task's pte.
- `task_struct/pte` migration can be used to find the task's pte.
- `task_struct/pte/pte` can be used to find the task's pte.

11. Multicore Processor Scheduling

Pseudo Code

```
// Multicore(4 2 2 2) 4x2x2x2
// - 4: 4 LLC, SMT 2x2, 2x/2 4x2x2
def multicore_scheduler(tasks, cores, topology):
    # topology: L3 4x2x2, SMT sibling 2x2
    while has_runnable(tasks):
        for task in ready_queue(tasks):
            # 1) 4 2x2 2x2(4 warm)
            candidate = task.last_core

            # 2) 4 LLC 4 2x2 2x2 4x2x2
            if not is_eligible(candidate):
                candidate = pick_idle_core_same_llc(task, topology)

            # 3) SMT sibling 4x2x2(4 CPU-bound task 4x2x2)
            candidate = avoid_heavy_smt_pair(candidate, task, topology)

        dispatch(task, candidate)
```

4x2x2

Multicore Processor Scheduling 4 Multiple-Processor 4 4x2x2, "4x2x2 CPU 4x2x2 4x2x2" 4x2x2 4x2x2 4x2x2. 4x2x2 4x2x2 4x2x2.

- 4x2x2(LLC) 4x2x2 4x2x2
- SMT(4x2x2x2) sibling 4x2x2 4x2x2
- 4x2x2 packing/spreading 4x2x2 4x2x2-4x2x2 4x2x2

Case Study

Task	Type	Burst	4x2x2
A	CPU-bound	20	4x2x2 4x2x2

B	CPU-bound	18	
C	Cache-sensitive	10	L3
D	I/O-mixed	8	

1

4

(2 SMT)

(C0,C1), (C2,C3)

(C0, C1)

(C2, C3)

- A -> C0

- B -> C2

- C -> C1 (A LLC)

- D -> C3 (I/O SMT)

- CPU-bound A, B SMT

- Cache-sensitive C SMT

Multicore

"SMT"

"LLC"

A	C0	CPU-bound
B	C2	A SMT SMT
C	C1	A LLC

D	C3	I/O ■■ ■■■■■ sibling ■■ ■■■ ■■■■■ ■■■
---	----	--

11/11/2019

■ ■	■ ■
SMT ■ ■ ■ ■	A ■ B ■ ■ ■ ■ ■ ■ ■ ■ ■ ■
LLC ■ ■ ■ ■	C ■ C0/C1 ■ ■ ■ ■ ■ ■ miss ■ ■ ■ ■
■ ■ ■ ■ ■ ■	■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■

THESE RESEARCH RESULTS ARE THE PROPERTY OF THE U.S. GOVERNMENT, AND ARE NOT TO BE REPRODUCED OR TRANSMITTED IN ANY FORM OR BY ANY MEANS, ELECTRONIC OR MECHANICAL, INCLUDING PHOTOCOPYING, RECORDING, OR BY ANY INFORMATION STORAGE AND RETRIEVAL SYSTEM, WITHOUT PERMISSION FROM THE U.S. GOVERNMENT.



- **■■■ ■■■ ■■■**: arrival time, CPU burst time, core topology(LLC/SMT), affinity, task class
- **■■■ ■■■■**: **■■■ ■■■■■** cache locality **■** sibling contention **■ ■■■■ ■■■■**
- **■■■ ■■■■**: **■■■** packing/spreading **■■■**, thermal/power **■■■ ■■■**

11

- 22

- ## Multiple-Processor vs Multicore

■ ■	Multiple-Processor Scheduling	Multicore Processor Scheduling
-----	-------------------------------	--------------------------------

	 (/))	 ()
	, , NUMA 	SMT , (LLC)
migration 	vs 	warm vs sibling
	work stealing, NUMA-aware balancing	core packing/spreading, LLC-aware placement
	/ , 	/

 , Multicore Scheduling Multiple-Processor Scheduling .
" " , " " .

12. Real-Time Scheduling

Pseudo Code

```
// Real-Time Scheduling     (Rate Monotonic Scheduling)
def rate_monotonic_scheduler(processes):
    #     (period)     (    )
    processes.sort(key=lambda p: p.period)

    while has_runnable_or_waiting_process(processes):
        now = current_time()
        #       
runnable_processes = [p for p in processes if p.arrival_time <= now and not p.is_completed()]
        if runnable_processes:
            next_process = min(runnable_processes, key=lambda p: p.period) #       
            execute(next_process) #     
```

```
// Real-Time Scheduling     (Earliest Deadline First)
def edf_scheduler(processes):
```

```
while has_runnable_or_waiting_process(processes):
    now = current_time()
    # Find all processes that are ready to run (arrival_time <= now and not p.is_completed())
    runnable_processes = [p for p in processes if p.arrival_time <= now and not p.is_completed()]
    if runnable_processes:
        next_process = min(runnable_processes, key=lambda p: p.deadline) # Select process with earliest deadline
        execute(next_process) # Execute the selected process
```

Real-Time Scheduling

Real-Time Scheduling **is a scheduling algorithm that takes into account the deadline of each process.** **It ensures that all processes meet their deadlines.** **It is used in systems where missing a deadline can have serious consequences.** **Examples of real-time scheduling algorithms include Rate Monotonic Scheduling (RMS), Earliest Deadline First (EDF), and Least Laxity First (LLF).** **Rate Monotonic Scheduling (RMS) is a static scheduling algorithm that assigns priorities to processes based on their periods.** **Earliest Deadline First (EDF) is a dynamic scheduling algorithm that assigns priorities to processes based on their deadlines.** **Least Laxity First (LLF) is a dynamic scheduling algorithm that assigns priorities to processes based on their laxity (the difference between their deadline and their current time).**

Case Study

Process	Arrival Time	Burst Time	Period	Deadline
P1	0	3	8	8
P2	0	2	5	5
P3	1	2	10	10
P4	2	1	4	4

Processes: P1, P2, P3, P4 **Arrival Times:** P1 (0), P2 (0), P3 (1), P4 (2) **Periods:** P1 (8), P2 (5), P3 (10), P4 (4) **Deadlines:** P1 (8), P2 (5), P3 (10), P4 (4) **Scheduling Algorithm:** Rate Monotonic Scheduling (RMS) **Priority Assignment:** P1 (1), P2 (2), P3 (3), P4 (4) **Execution Order:** P1, P2, P3, P4

Pseudo Execution Timeline Analysis

Timeline:

Time	Processes in Queue	Process Executing
t=0	P1, P2	P1, P2
t=1	P1(1), P2(1), P3(1)	P3

t=2	P1(■■■■), P2(■■■), P3(■■■), P4 ■■■	P4 ■■■
t=3	P2 ■■■ ■■■	P1 ■■■, P2 ■■■ ■
t=5	P2, P4 ■■■	■■■ ■■ ■■■■

RMS (Rate Monotonic Scheduling) - ■■■ ■■■■■ ■■■

■■■■■ ■■: ■■■ ■■■■■ ■■ ■■■■■

Process	Period	RMS Priority
P4	4	1 (■■■)
P2	5	2
P1	8	3
P3	10	4 (■■■)

Gantt Chart for RMS Schedule

P2	P4	P1	P4	P2	P4	P3	P1	
3	6	7 0 9		10		12		15

RMS ■■■ ■■■ ■■■

■■■	■■■	■■■■■ ■■■	■■■
0~2	P2	P1(period=8), P2(period=5) ■■■ → P2 ■■■	P2 ■■■

2~3	P4	P4(period=4) ■■ → P2 ■■	P4 ■■, P2 ■■
3~6	P1	P4 ■■ ■, ■■ ■■■■ ■ period ■■ P2(rem=3) ■■■ ■■■ P1 ■■	P1 ■■ (P2 ■■ t=5)
6~7	P4	P4 ■■ ■■(t=4,8→■■), ■■ P4(period=4, 2nd instance) ■■	P4 ■■■
7~9	P2	P2 ■■ ■■(t=5,10), P2(period=5, 2nd instance) ■■	P2 ■■■
9~10	P4	P4(period=4, 3rd instance) ■■	P4 3■■ ■■■■

10~12	P3	P3(period=10) ■■ → period■■ ■■ ■■■ ■■ ■■■■ ■■ ■ ■■	P3 ■■
12~15	P1	P1(period=8, 2nd instance) ■■ → 2■■■ ■■■ P1 ■■	P1 2■■■ ■■

RMS ■■ ■ ■■■■■ ■■ ■■

Process	■■ ■■	Deadline	Deadline Miss?	■■
P1 (1st)	6	8	✓ ■■	t=6 ≤ 8
P2 (1st)	5	5	✓ ■■	t=5 = 5 (■■ ■■)
P3 (1st)	12	10	✗ ■■	t=12 > 10 (Deadline Miss!)
P4 (1st)	3	4	✓ ■■	t=3 ≤ 4
P4 (2nd)	7	8	✓ ■■	t=7 ≤ 8

P4 (3rd)	10	12	✓ ■■	t=10 ≤ 12
-------------	----	----	---------	-----------------

EDF (Earliest Deadline First) - ■■ ■■■■■ ■■

■■■■ ■■: ■■■■■ ■■■■■ ■■ ■■■■ (■ ■■■ ■■■)

Gantt Chart for EDF Schedule

P2	P1	P4	P1	P2	P3	P4	P1	P4	
0 4	5	7	0 9	11	12 14				15

EDF ■■ ■■ ■■

■■	■■	■■■■ ■■	■■
0~2	P2	P1 deadline=8, P2 deadline=5 → P2 ■■ (■ ■■)	P2 ■■
2~4	P1	P2 ■■ ■, P1 deadline=8, P3 deadline=10 → P1 ■■	P1 ■■
4~5	P4	P4 ■■ deadline=4 → ■■ ■■ ■■	P4 ■■

5~7	P1	P4 ■■ ■, P1 deadline=8, P3 deadline=10 → P1 ■■	P1 ■■
7~9	P2	P2 ■■ ■■ deadline=10 (2nd), P1 deadline=8 ■■ ■■→P2 ■■, P1 ■■ 1ms	P2 ■■■
9~11	P3	P3 deadline=10 (■■ ■■■■■ ■■■ ■■) → P1 ■■ ■■ ■■, P2 ■■ ■ P3 ■■	P3 ■■

11~12	P4	P4 ■ ■ ■ ■ deadline=8 (2nd, t=2+4=6 ■ ■ ■ ■ 8) → ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■	P4 ■ ■ ■
12~14	P1	P1 ■ ■ ■ ■ deadline=16 (2nd) → ■ ■ ■ ■ ■ ■ ■ ■ ■	P1 2 ■ ■ ■ ■
14~15	P4	P4 deadline (3rd instance) ■ ■ ■ ■	P4 ■ ■ ■ ■

EDF ■ ■ ■ ■ ■ ■ ■ ■ ■ ■

Process	■ ■ ■ ■	Deadline	Deadline Miss?	■ ■
P1 (1st)	7	8	✓ ■ ■	t=7 ≤ 8
P2 (1st)	2	5	✓ ■ ■	t=2 ≤ 5

P3 (1st)	11	10	X ■■	t=11 > 10 (Deadline Miss!)
P4 (1st)	5	4	X ■■	t=5 > 4 (Deadline Miss!)
P4 (2nd)	12	8	X ■■	t=12 > 8 (Deadline Miss!)
P4 (3rd)	15	12	X ■■	t=15 > 12 (Deadline Miss!)

■■ ■■ ■ ■ ■■

■ ■■■ 4■■ ■■■■■■ ■■ ■■ ■■■ ■■■■■■ ■■ ■, RMS■ EDF■ ■■ ■■ ■■■■ ■■■■
Deadline Miss ■■ ■■■ ■■■■ ■■ ■■■■■■ ■■■■■■.

■■ ■■: RMS vs EDF

1. ■■■■ ■■ ■■■ ■■

■■	RMS	EDF
■■■■ ■■	■■(Static) - ■■ ■■	■■(Dynamic) - ■ ■■ ■■■
■■ ■■	■■■■ ■■(Period)	■■ ■■■■(Deadline)
■■ ■■	■■■■ ■■ ■ 1■■	■ ■■■■ ■■■■ ■■

- EDF scheduling: $\frac{1}{\text{period}}$ (deadline-driven scheduling)

- $\text{deadline} \leq \text{period}$, $\text{period} \leq \text{deadline}$

EDF scheduling: $\text{RMS} \leq \frac{1}{\text{period}}$, $\text{period} \leq \text{deadline}$ (period, deadline) $\leq \frac{1}{\text{period}}$. EDF scheduling is a deadline-driven scheduling algorithm.

EDF

- EDF scheduling: arrival time, CPU burst time, period, deadline $\leq \text{period}$, $\text{period} \leq \text{deadline}$.

- EDF scheduling: $\text{period} \leq \text{deadline}$ (period) $\leq \text{deadline}$ (deadline) $\leq \text{period}$, $\text{period} \leq \text{deadline}$.

- EDF scheduling: $\text{period} \leq \text{deadline}$ (period) $\leq \text{deadline}$ (deadline) $\leq \text{period}$, $\text{period} \leq \text{deadline}$.

EDF

- 2019 年 12 月 31 日，本公司 2019 年度财务报表已经审计。

- 2019 年 12 月 31 日，本公司 2019 年度财务报表已经审计。

- 2019 年 12 月 31 日，本公司 2019 年度财务报表已经审计。

■ ■

- 2019 年 12 月 31 日，本公司 2019 年度财务报表已经审计。

- 2019 年 12 月 31 日，本公司 2019 年度财务报表已经审计。

- 2019 年 12 月 31 日，本公司 2019 年度财务报表已经审计。